Spatial Cluster Detection Applied to Boston

Rémi Lahay Lucien Mirlicourtois Anas Naji Alexandre Picart

2026-03-11

Les packages:

```
library(spData)
```

```
## Warning: le package 'spData' a été compilé avec la version R 4.4.3
```

```
## To access larger datasets in this package, install the spDataLarge
## package with: `install.packages('spDataLarge',
## repos='https://nowosad.github.io/drat/', type='source')`
```

```
library(sf)
```

```
## Warning: le package 'sf' a été compilé avec la version R 4.4.3
```

```
## Linking to GEOS 3.13.0, GDAL 3.10.1, PROJ 9.5.1; sf_use_s2() is TRUE
```

```
library(dplyr)
```

```
## Warning: le package 'dplyr' a été compilé avec la version R 4.4.3
```

```
##
## Attachement du package : 'dplyr'
```

```
## Les objets suivants sont masqués depuis 'package:stats':
##
##     filter, lag
```

```
## Les objets suivants sont masqués depuis 'package:base':
##
##     intersect, setdiff, setequal, union
```

```
library(leaflet)
```

```
## Warning: le package 'leaflet' a été compilé avec la version R 4.4.3
```

```r
library(spdep)
```

```
## Warning: le package 'spdep' a été compilé avec la version R 4.4.3
```

```r
library(ggplot2)
```

```
## Warning: le package 'ggplot2' a été compilé avec la version R 4.4.3
```

```r
library(rsatscan)
```

```
## Warning: le package 'rsatscan' a été compilé avec la version R 4.4.3

## rsatscan only does anything useful if you have SaTScan

## See http://www.satscan.org/ for free access
```

```r
library(spatialreg)
```

```
## Warning: le package 'spatialreg' a été compilé avec la version R 4.4.3

## Le chargement a nécessité le package : Matrix

##
## Attachement du package : 'spatialreg'

## Les objets suivants sont masqués depuis 'package:spdep':
##
##     get.ClusterOption, get.coresOption, get.mcOption,
##     get.VerboseOption, get.ZeroPolicyOption, set.ClusterOption,
##     set.coresOption, set.mcOption, set.VerboseOption,
##     set.ZeroPolicyOption
```

Données

```r
data(boston)
```

Exploration des données

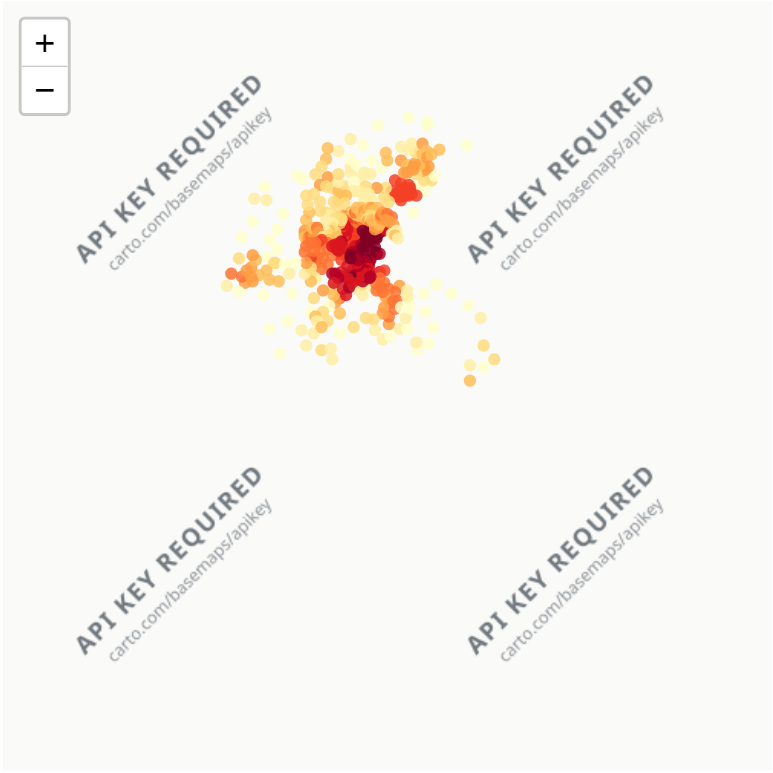
Scan test sur le CRIM

```r
# df doit être en WGS84 pour leaflet
if (is.na(st_crs(df))) df <- st_set_crs(df, 4326)
df_wgs <- st_transform(df, 4326)

# variable à cartographier
df_wgs$crim_log <- log1p(df_wgs$CRIM)

# palette en classes (quantiles) => nuances plus visibles
bins <- quantile(df_wgs$crim_log, probs = seq(0, 1, by = 0.1), na.rm = TRUE)
pal  <- colorBin(palette = "YlOrRd", domain = df_wgs$crim_log, bins = bins, na.color = "#BDBDBD")

leaflet(df_wgs) %>%
  addProviderTiles(providers$CartoDB.PositronNoLabels) %>%
  addCircleMarkers(
    radius = 4,
    stroke = FALSE,
    fillOpacity = 0.85,
    color = ~pal(crim_log),
    popup = ~paste0("CRIM: ", signif(CRIM, 4), "<br>log1p(CRIM): ", signif(crim_log, 4))
  ) %>%
  addLegend("bottomright", pal = pal, values = ~crim_log, title = "log1p(CRIM)")
```




```
# vérifie
exists("ssenv", inherits = FALSE)
```

```
## [1] TRUE
```

```
# recharge le package (recrée ssenv)
detach("package:rsatscan", unload = TRUE)  # si déjà attaché
library(rsatscan)
```

```
## Warning: le package 'rsatscan' a été compilé avec la version R 4.4.3

## rsatscan only does anything useful if you have SaTScan

## See http://www.satscan.org/ for free access
```

```
# maintenant ça doit marcher
invisible(ss.options(reset = TRUE))
```

```
# CRS : pour ellipse => cartésien (mètres)
if (is.na(st_crs(df))) df <- st_set_crs(df, 4326)
if (isTRUE(st_is_longlat(st_crs(df)))) df <- st_transform(df, 3857)

# variable continue (recommandé: log + standardisation)
y <- log1p(df$CRIM)
y <- as.numeric(scale(y))

# enlever NA/inf
ok <- is.finite(y)
df <- df[ok, ]
y <- y[ok]

# IMPORTANT : locid créé APRÈS filtrage
df$locid <- seq_len(nrow(df))
xy <- st_coordinates(df)

# --- 1) écrire fichiers SaTScan ---
td <- file.path(tempdir(), "satscan_crim")
dir.create(td, showWarnings = FALSE, recursive = TRUE)

geo <- data.frame(locid = df$locid, X = xy[,1], Y = xy[,2])
write.table(geo, file.path(td, "crim.geo"),
            sep = " ", row.names = FALSE, col.names = FALSE, quote = FALSE)

options(OutDec = ".")
cas <- data.frame(locid = df$locid, n = 1, y = y)
write.table(cas, file.path(td, "crim.cas"),
            sep = " ", row.names = FALSE, col.names = FALSE, quote = FALSE)

# --- 2) paramètres SaTScan ---
# si ss.options() plante avec ssenv: detach("package:rsatscan", unload=TRUE); library(rsatscan)
invisible(ss.options(reset = TRUE))

ss.options(list(
  CaseFile = "crim.cas",
  CoordinatesFile = "crim.geo",
```

```
  CoordinatesType = 0,                 # cartésien
  PrecisionCaseTimes = 0,              # pas de date/temps
  AnalysisType = 1,                    # purely spatial
  ModelType = 5,                       # Normal model (continuous)
  ScanAreas = 3,                       # high OR low
  MonteCarloReps = 999,
  MaxSpatialSizeInPopulationAtRisk = 50,
  SpatialWindowShapeType = 1,          # ellipse
  # évite clusters de 1 point
  MinimumCasesInHighRateClusters = 20,
  MinimumCasesInLowRateClusters  = 20,
  ResultsFile = "crim_scan",
  LogRunToHistoryFile = "n"
))
write.ss.prm(td, "crim_scan")

# --- 3) run SaTScan ---
# Windows:
ssloc <- "C:/Program Files/SaTScan"
batch <- "SaTScanBatch64"

res_crim <- satscan(
  prmlocation = td,
  prmfilename = "crim_scan",
  sslocation = ssloc,
  ssbatchfilename = batch,
  verbose = TRUE
)
```

```
##
##
##  Any following message is from st_read() in the sf package
```

```
summary(res_crim)
```

```
## Purely Spatial analysis
## scanning for clusters with high or low values
## using the Normal model.
##
## Study period.....................: 2000/1/1 to 2000/12/31
## Number of locations..............: 506
## Total number of cases............: 506
## Mean.............................: 0.00000000000000024
## Variance.........................: 1.00
## ----------------------------------------------------------------------------------------------
## There were 2 clusters identified.
## There were 2 clusters with p < .05.
```

```
names(res_crim)
```

```
## [1] "main"       "col"        "rr"         "gis"        "llr"
## [6] "sci"        "shapeclust" "prm"
```

```r
summary(res_crim)        # résumé lisible
```

```
## Purely Spatial analysis
## scanning for clusters with high or low values
## using the Normal model.
##
## Study period.....................: 2000/1/1 to 2000/12/31
## Number of locations..............: 506
## Total number of cases............: 506
## Mean.............................: 0.00000000000000024
## Variance.........................: 1.00
## ________________________________________________________________________________________
## There were 2 clusters identified.
## There were 2 clusters with p < .05.
```

```r
names(res_crim)          # voir les tables dispo
```

```
## [1] "main"       "col"        "rr"         "gis"        "llr"
## [6] "sci"        "shapeclust" "prm"
```

```r
res_crim$col             # table des clusters (souvent la plus utile)
```

```
##   CLUSTER LOC_ID        X       Y   E_MINOR   E_MAJOR E_ANGLE E_SHAPE     SPAN
## 1       1    429 -7909484 5190084  4596.161  6894.242      45     1.5 12558.17
## 2       2     67 -7911365 5213929 17056.513 68226.051      30     4.0 55507.67
##   START_DATE   END_DATE NUMBER_LOC       LLR TEST_STAT P_VALUE OBSERVED
## 1   2000/1/1 2000/12/31        151 315.88595 309.50376   0.001      151
## 2   2000/1/1 2000/12/31        232  78.56266  62.85012   0.001      232
##      MEAN_IN   MEAN_OUT  VARIANCE       STD GINI_CLUST
## 1  1.2934989 -0.5501925 0.2869174 0.5356467      FALSE
## 2 -0.5609272  0.4749457 0.7330625 0.8561907      FALSE
```

```r
res_crim$sci             # infos complémentaires (selon sorties activées)
```

```
##   CLUSTER DATASET CATEGORY OBSERVED    MEAN_IN   MEAN_OUT  VARIANCE       STD
## 1       1       1        1      151  1.2934989 -0.5501925 0.2869174 0.5356467
## 2       2       1        1      232 -0.5609272  0.4749457 0.7330625 0.8561907
```

```r
df$locid <- seq_len(nrow(df))

gis_crim <- res_crim$gis

loc_col <- names(gis_crim)[grepl("loc", names(gis_crim), ignore.case = TRUE) & grepl("id", names(gis_cri
clu_col <- names(gis_crim)[grepl("cluster", names(gis_crim), ignore.case = TRUE)][1]

if (is.na(loc_col) || is.na(clu_col)) {
  stop("Je ne trouve pas les colonnes locid/cluster dans res$gis. Fais names(res$gis) pour voir les noms
}

df$cluster_id <- gis_crim[[clu_col]][match(df$locid, gis_crim[[loc_col]])]
```

```r
df$cluster_id[is.na(df$cluster_id)] <- 0L

table(df$cluster_id)
```

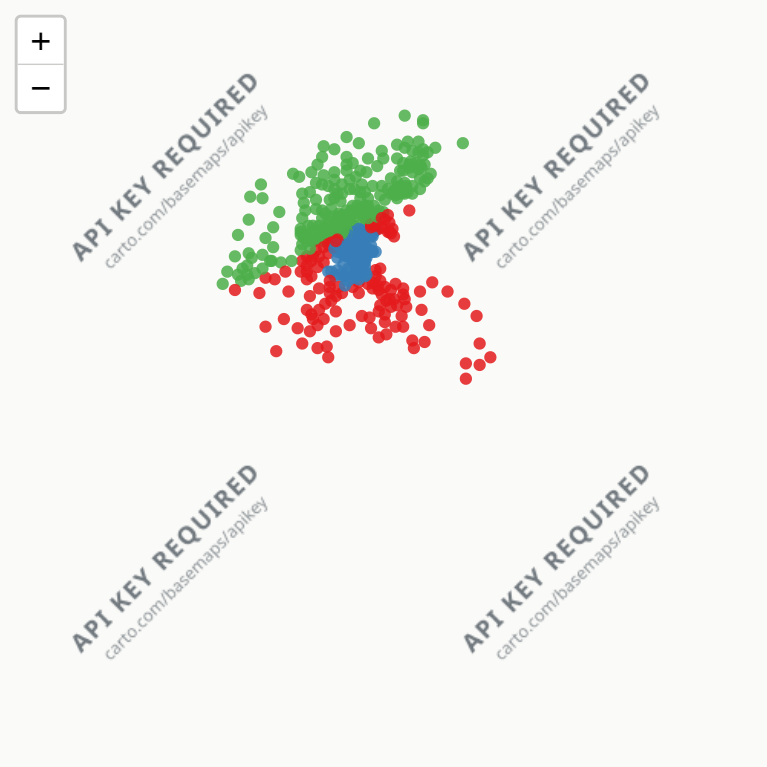
```
##
##   0   1   2
## 123 151 232
```

```r
# si CRS manquant, définis-le avant st_transform
# sf0 <- st_set_crs(sf0, 4326)  # si déjà lon/lat
df <- st_transform(df, 4326)

df$cluster <- ifelse(df$cluster == 0, "0_out", paste0("C", df$cluster))
df$cluster <- factor(df$cluster)

pal <- colorFactor("Set1", domain = df$cluster)

leaflet(df) %>%
  addProviderTiles(providers$CartoDB.Positron) %>%
  addCircleMarkers(
    radius = 4,
    stroke = FALSE,
    fillOpacity = 0.85,
    color = ~pal(cluster),
    popup = ~paste0("locid: ", locid, "<br>cluster: ", cluster)
  ) %>%
  addLegend("bottomright", pal = pal, values = ~cluster, title = "Clusters")
```


+
−
API KEY REQUIRED
carto.com/basemaps/apikey
API KEY REQUIRED
carto.com/basemaps/apikey
API KEY REQUIRED
carto.com/basemaps/apikey
API KEY REQUIRED
carto.com/basemaps/apikey
Clusters
0_out
C1
C2
Leaflet | © OpenStreetMap contributors © CARTO

Most Likely Clusters :

```
names(res_crim$col)
```

```
##  [1] "CLUSTER"    "LOC_ID"     "X"          "Y"          "E_MINOR"
##  [6] "E_MAJOR"    "E_ANGLE"    "E_SHAPE"    "SPAN"       "START_DATE"
## [11] "END_DATE"   "NUMBER_LOC" "LLR"        "TEST_STAT"  "P_VALUE"
## [16] "OBSERVED"   "MEAN_IN"    "MEAN_OUT"   "VARIANCE"   "STD"
## [21] "GINI_CLUST"
```

```
# --- tables SaTScan
col_tbl <- res_crim$col
gis_tbl <- res_crim$gis

# --- 1) Most Likely Cluster = 1re ligne
mlc_cluster <- col_tbl$CLUSTER[1]

# --- 2) LOC_ID du MLC
mlc_locid <- unique(gis_tbl$LOC_ID[gis_tbl$CLUSTER == mlc_cluster])

if (length(mlc_locid) == 0) {
  stop("Aucun LOC_ID trouvé pour le Most Likely Cluster.")
}

# --- 3) fichier d'exclusion
exclude_file <- file.path(td, "exclude.txt")

write.table(
  mlc_locid,
  file = exclude_file,
  row.names = FALSE,
  col.names = FALSE,
  quote = FALSE
)

readLines(exclude_file, n = 10)
```

```
##  [1] "429" "478" "427" "479" "428" "417" "430" "416" "431" "425"
```

```
# --- 4) vérifier quels fichiers existent vraiment dans td
list.files(td)
```

```
## [1] "crim.cas"      "crim.geo"      "crim_scan.prm" "exclude.txt"
```

```
# adapte ici si tes fichiers s'appellent house.cas / house.geo
case_file_name <- "crim.cas"
geo_file_name  <- "crim.geo"
```

```r
# --- 5) paramètres standards
ss.options(list(
  CaseFile = "C:/Users/lmirl/AppData/Local/Temp/RtmpkvV0Pk/satscan_crim/crim.cas",
  CoordinatesFile = "C:/Users/lmirl/AppData/Local/Temp/RtmpkvV0Pk/satscan_crim/crim.geo",
  CoordinatesType = 0,
  PrecisionCaseTimes = 0,
  AnalysisType = 1,
  ModelType = 5,
  ScanAreas = 3,
  MonteCarloReps = 999,
  MaxSpatialSizeInPopulationAtRisk = 50,
  SpatialWindowShapeType = 1,
  MinimumCasesInHighRateClusters = 20,
  MinimumCasesInLowRateClusters = 20,
  ResultsFile = "crim_secondary",
  LogRunToHistoryFile = "n"
))

# --- 6) paramètres extra : ExcludeFile
ss.options.extra(list(
  ExcludeFile = "exclude.txt"
))

write.ss.prm(td, "crim_secondary")

# --- 7) relancer SaTScan
res_secondary <- satscan(
  prmlocation = td,
  prmfilename = "crim_secondary",
  sslocation = ssloc,
  ssbatchfilename = batch,
  verbose = TRUE
)
```

```
##
##
##  Any following message is from st_read() in the sf package
```

```r
summary(res_secondary)
```

```
## NA
## NA
## NA
## NA
## NA
## NA
## NA
## NA
## NA
## NA
## No cluster dBase file detected.
## Run SaTScan with 'MostLikelyClusterEachCentroidDBase' parameter set to 'y' for cluster information.
```

```r
# vérifier les vrais fichiers présents
list.files(td)
```

```
## [1] "crim.cas"           "crim.geo"           "crim_scan.prm"
## [4] "crim_secondary.prm" "exclude.txt"
```

```r
case_file <- normalizePath(file.path(td, "crim.cas"), winslash = "\\", mustWork = TRUE)
geo_file  <- normalizePath(file.path(td, "crim.geo"),  winslash = "\\", mustWork = TRUE)
excl_file <- normalizePath(file.path(td, "exclude.txt"), winslash = "\\", mustWork = TRUE)

case_file
```

```
## [1] "C:\\Users\\lmirl\\AppData\\Local\\Temp\\RtmpI3eCcG\\satscan_crim\\crim.cas"
```

```r
geo_file
```

```
## [1] "C:\\Users\\lmirl\\AppData\\Local\\Temp\\RtmpI3eCcG\\satscan_crim\\crim.geo"
```

```r
excl_file
```

```
## [1] "C:\\Users\\lmirl\\AppData\\Local\\Temp\\RtmpI3eCcG\\satscan_crim\\exclude.txt"
```

```r
summary(res_secondary)
```

```
## NA
## NA
## NA
## NA
## NA
## NA
## NA
## NA
## NA
## NA
## No cluster dBase file detected.
## Run SaTScan with 'MostLikelyClusterEachCentroidDBase' parameter set to 'y' for cluster information.
```

```r
summary(res_crim)
```

```
## Purely Spatial analysis
## scanning for clusters with high or low values
## using the Normal model.
##
## Study period.....................: 2000/1/1 to 2000/12/31
## Number of locations..............: 506
## Total number of cases............: 506
## Mean.............................: 0.00000000000000024
## Variance.........................: 1.00
## ____________________________________________________________________________________________
## There were 2 clusters identified.
## There were 2 clusters with p < .05.
```

Modèle OLS :

```
yvar  <- "CMEDV"
xvars <- c("CRIM", "ZN", "INDUS", "AGE", "TAX", "PTRATIO")


fml <- as.formula(paste(yvar, "~", paste(xvars, collapse = " + ")))
m0 <- lm(fml, data = df)

# résidus (tu peux aussi mettre rstandard(m0))
df$resid0 <- residuals(m0)
df$resid0 <- as.numeric(scale(df$resid0))  # optionnel mais souvent pratique
```

```
summary(m0)
```

```
##
## Call:
## lm(formula = fml, data = df)
##
## Residuals:
##     Min      1Q  Median      3Q     Max
## -13.290  -4.239  -1.259   2.501  34.081
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept) 54.115980   3.354925  16.130  < 2e-16 ***
## CRIM        -0.171378   0.046439  -3.690 0.000249 ***
## ZN           0.016945   0.018587   0.912 0.362387
## INDUS       -0.277802   0.080746  -3.440 0.000630 ***
## AGE         -0.019591   0.016330  -1.200 0.230836
## TAX         -0.001726   0.003284  -0.526 0.599353
## PTRATIO     -1.409794   0.177282  -7.952 1.23e-14 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 7.248 on 499 degrees of freedom
## Multiple R-squared:  0.3844, Adjusted R-squared:  0.377
## F-statistic: 51.93 on 6 and 499 DF,  p-value: < 2.2e-16
```

Test de Moran

```
coords <- st_coordinates(df)
knn <- knearneigh(coords, k = 8)
nb <- knn2nb(knn)
listw <- nb2listw(nb, style = "W")
```

```
moran.test(df$resid0, listw)
```

```
##
##  Moran I test under randomisation
##
```

```
## data:  df$resid0
## weights: listw
##
## Moran I statistic standard deviate = 18.996, p-value < 2.2e-16
## alternative hypothesis: greater
## sample estimates:
## Moran I statistic       Expectation          Variance
##      0.3900427310     -0.0019801980      0.0004259026
```

```
for (k in c(5, 8, 10, 15)) {
  knn <- knearneigh(coords, k = k)
  nb <- knn2nb(knn)
  lw <- nb2listw(nb, style = "W")
  print(k)
  print(moran.test(df$resid0, lw))
}
```

```
## [1] 5
##
##  Moran I test under randomisation
##
## data:  df$resid0
## weights: lw
##
## Moran I statistic standard deviate = 18.004, p-value < 2.2e-16
## alternative hypothesis: greater
## sample estimates:
## Moran I statistic       Expectation          Variance
##      0.4676163738     -0.0019801980      0.0006803068
##
## [1] 8
##
##  Moran I test under randomisation
##
## data:  df$resid0
## weights: lw
##
## Moran I statistic standard deviate = 18.996, p-value < 2.2e-16
## alternative hypothesis: greater
## sample estimates:
## Moran I statistic       Expectation          Variance
##      0.3900427310     -0.0019801980      0.0004259026
##
## [1] 10
##
##  Moran I test under randomisation
##
## data:  df$resid0
## weights: lw
##
## Moran I statistic standard deviate = 19.419, p-value < 2.2e-16
## alternative hypothesis: greater
## sample estimates:
## Moran I statistic       Expectation          Variance
```

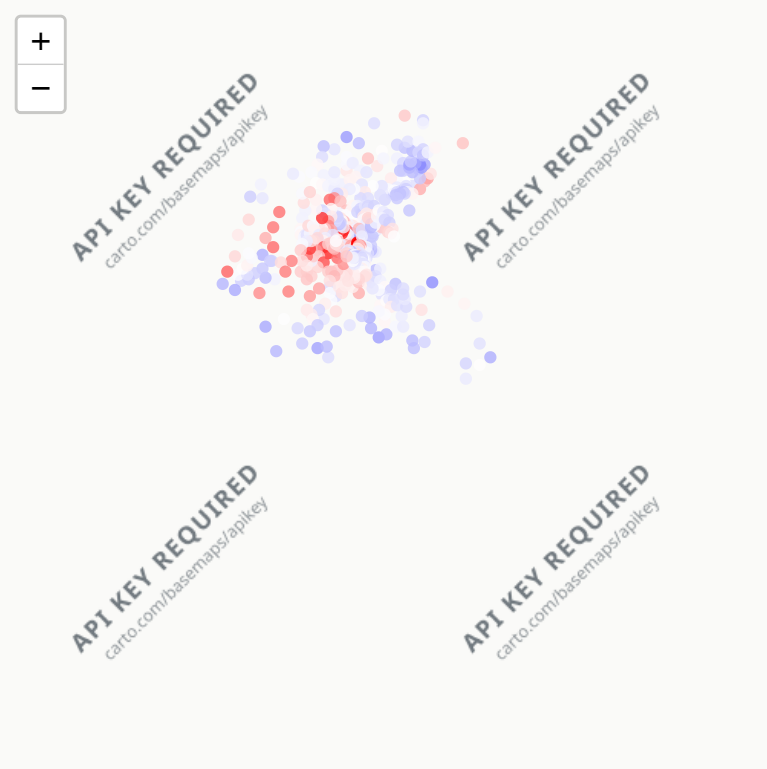
```
##      0.3556313748     -0.0019801980      0.0003391316
##
## [1] 15
##
##  Moran I test under randomisation
##
## data:  df$resid0
## weights: lw
##
## Moran I statistic standard deviate = 19.828, p-value < 2.2e-16
## alternative hypothesis: greater
## sample estimates:
## Moran I statistic       Expectation          Variance
##      0.2938885938     -0.0019801980      0.0002226542
```

```r
m <- st_transform(df, 4326)

lim <- 4
m$resid0_clip <- pmax(pmin(m$resid0, lim), -lim)

pal <- colorNumeric(
  palette = colorRampPalette(c("blue", "white", "red"))(256),
  domain = c(-lim, lim),
  na.color = "#BDBDBD"
)

leaflet(m) %>%
  addProviderTiles(providers$CartoDB.PositronNoLabels) %>%
  addCircleMarkers(
    radius = 4,
    stroke = FALSE,
    fillOpacity = 0.8,
    color = ~pal(resid0_clip),
    popup = ~paste0(
      "resid0: ", signif(resid0, 4),
      "<br>resid0 (clip): ", signif(resid0_clip, 4)
    )
  ) %>%
  addLegend(
    position = "bottomright",
    pal = pal,
    values = ~resid0_clip,
    title = "Résidus (clip [-4, 4])"
  )
```



Résidus (clip [-4, 4])

- -1
- 0
- 1
- 2
- 3
- 4

Leaflet | © OpenStreetMap contributors © CARTO

```
# 1) IMPORTANT : ellipse => coordonnées cartésiennes (pas lon/lat)
if (isTRUE(sf::st_is_longlat(sf::st_crs(df)))) {
```

```r
  df <- sf::st_transform(df, 3857)
}

# 2) variable à scanner
y <- df$resid0

# 3) enlever les NA / valeurs non finies
df <- df[is.finite(df$resid0), ]
y  <- df$resid0

# 4) IDs directement dans df
df$locid <- seq_len(nrow(df))

# 5) coordonnées
xy <- sf::st_coordinates(df)

td <- file.path(tempdir(), "satscan_house")
dir.create(td, showWarnings = FALSE, recursive = TRUE)

geo2 <- data.frame(
  locid = df$locid,
  X = xy[, 1],
  Y = xy[, 2]
)

write.table(
  geo2,
  file = file.path(td, "house.geo"),
  sep = " ",
  row.names = FALSE,
  col.names = FALSE,
  quote = FALSE
)

readLines(file.path(td, "house.geo"), n = 5)
```

```
## [1] "1 -7898674.46923673 5199254.07468672"
## [2] "2 -7898117.87178276 5204143.3133027"
## [3] "3 -7896559.39891165 5203466.1914073"
## [4] "4 -7895668.84298531 5204970.97242127"
## [5] "5 -7895000.92604055 5205723.45252267"
```

```r
# 6) forcer le séparateur décimal à "."
options(OutDec = ".")

# 7) fichier cas : LocationID, n, y
cas <- data.frame(
  locid = df$locid,
  n = 1,
  y = df$resid0
)

write.table(
```

```
  cas,
  file = file.path(td, "house.cas"),
  sep = " ",
  row.names = FALSE,
  col.names = FALSE,
  quote = FALSE
)

readLines(file.path(td, "house.cas"), n = 5)
```

```
## [1] "1 1 -0.891141691174229" "2 1 -0.484357413834024" "3 1 1.28558539930815"
## [4] "4 1 1.04642007496481"   "5 1 1.45878951758109"
```

```
summary(res_m0)        # résumé lisible
```

```
## Purely Spatial analysis
## scanning for clusters with high or low values
## using the Normal model.
##
## Study period.....................: 2000/1/1 to 2000/12/31
## Number of locations..............: 506
## Total number of cases............: 506
## Mean.............................: -0.00000000000000012
## Variance.........................: 1.00
## ------------------------------------------------------------------------------------------
## There were 3 clusters identified.
## There were 3 clusters with p < .05.
```

```
names(res_m0)          # voir les tables dispo
```

```
## [1] "main"       "col"        "rr"         "gis"        "llr"
## [6] "sci"        "shapeclust" "prm"
```

```
res_m0$col             # table des clusters (souvent la plus utile)
```

```
##   CLUSTER LOC_ID        X       Y   E_MINOR    E_MAJOR E_ANGLE E_SHAPE
## 1       1    371 -7908382 5193150   727.228   1090.842       0     1.5
## 2       2    236 -7919046 5193105  7928.665  11892.997      90     1.5
## 3       3    356 -7887264 5165474 28640.309 143201.545      72     5.0
##        SPAN START_DATE   END_DATE NUMBER_LOC      LLR TEST_STAT P_VALUE
## 1  1480.911   2000/1/1 2000/12/31          5 62.14595  62.14595   0.001
## 2 20850.466   2000/1/1 2000/12/31        122 43.09486  43.09486   0.001
## 3 52409.116   2000/1/1 2000/12/31        200 27.81715  27.81715   0.018
##   OBSERVED    MEAN_IN    MEAN_OUT  VARIANCE       STD GINI_CLUST
## 1        5  4.6668784 -0.04657563 0.7822068 0.8844245      FALSE
## 2      122  0.7014184 -0.22284646 0.8433819 0.9183583      FALSE
## 3      200 -0.3987346  0.26061088 0.8958796 0.9465092      FALSE
```

```
res_m0$sci
```

```
##   CLUSTER DATASET CATEGORY OBSERVED    MEAN_IN    MEAN_OUT  VARIANCE       STD
## 1       1       1        1        5  4.6668784 -0.04657563 0.7822068 0.8844245
## 2       2       1        1      122  0.7014184 -0.22284646 0.8433819 0.9183583
## 3       3       1        1      200 -0.3987346  0.26061088 0.8958796 0.9465092
```

```r
gis_m0 <- res_m0$gis

loc_col <- names(gis_m0)[grepl("loc", names(gis_m0), ignore.case = TRUE) &
                         grepl("id", names(gis_m0), ignore.case = TRUE)][1]
clu_col <- names(gis_m0)[grepl("cluster", names(gis_m0), ignore.case = TRUE)][1]

if (is.na(loc_col) || is.na(clu_col)) {
  stop("Je ne trouve pas les colonnes locid/cluster dans res$gis. Fais names(res$gis) pour voir les noms
}

df$cluster_id_scan <- gis_m0[[clu_col]][match(df$locid, gis_m0[[loc_col]])]
df$cluster_id_scan[is.na(df$cluster_id_scan)] <- 0L

table(df$cluster_id_scan)
```

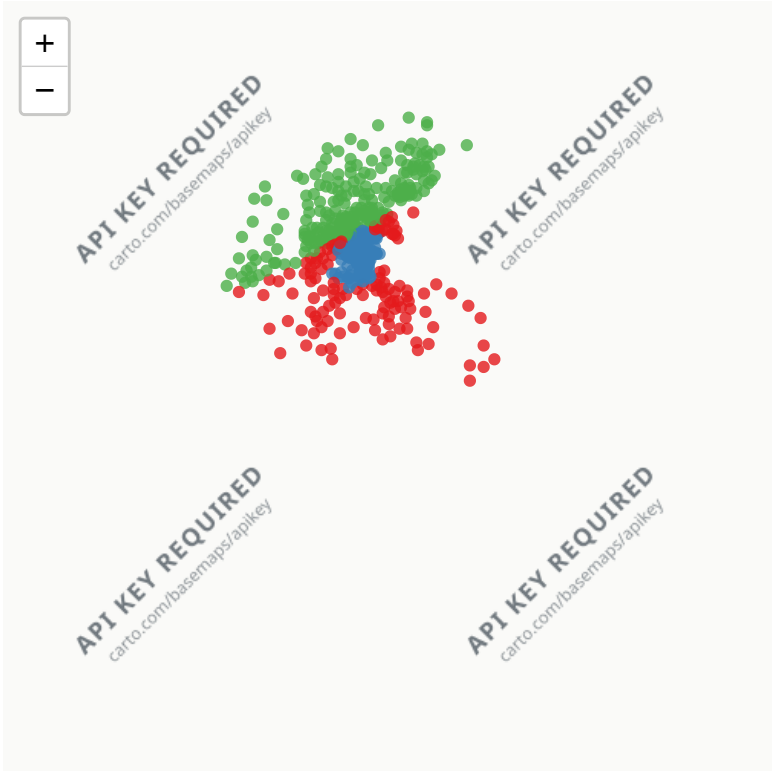
```
##
##   0   1   2   3
## 179   5 122 200
```

```r
df_wgs <- st_transform(df, 4326)

df_wgs$cluster <- ifelse(df_wgs$cluster_id == 0, "0_out", paste0("C", df_wgs$cluster_id))
df_wgs$cluster <- factor(df_wgs$cluster)

pal <- colorFactor("Set1", domain = df_wgs$cluster)

leaflet(df_wgs) %>%
  addProviderTiles(providers$CartoDB.PositronNoLabels) %>%
  addCircleMarkers(
    radius = 4,
    stroke = FALSE,
    fillOpacity = 0.8,
    color = ~pal(cluster),
    popup = ~paste0("locid: ", locid, "<br>cluster: ", cluster)
  ) %>%
  addLegend(
    "bottomright",
    pal = pal,
    values = ~cluster,
    title = "Clusters"
  )
```




```
df$cluster_id_scan <- factor(df$cluster_id_scan)
levels(df$cluster_id_scan)
```

```
## [1] "0" "1" "2" "3"
```

```
m1 <- lm(CMEDV ~ CRIM + ZN + INDUS + AGE + TAX + PTRATIO + cluster_id,
         data = df)

summary(m1)
```

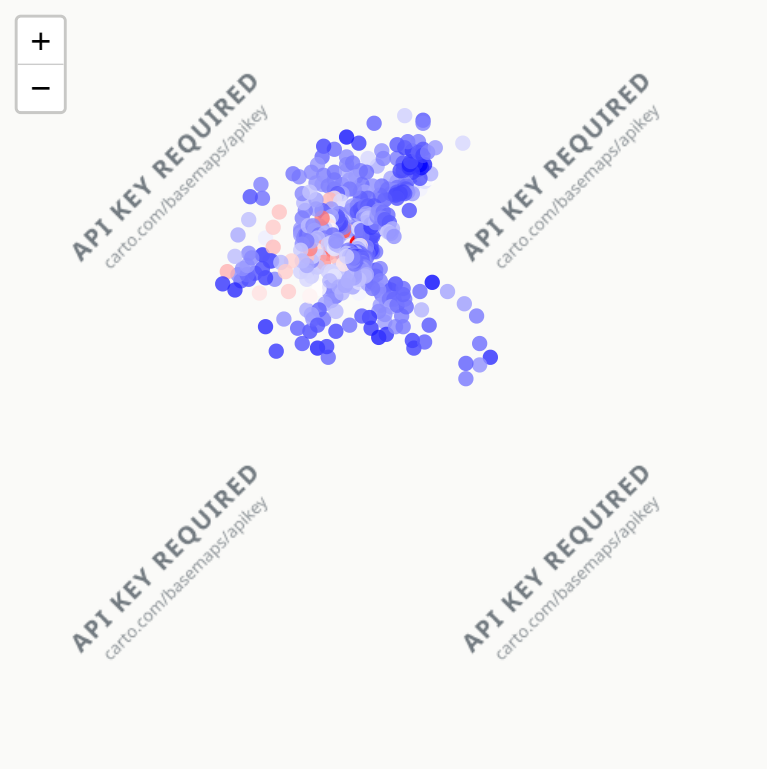
```
##
## Call:
## lm(formula = CMEDV ~ CRIM + ZN + INDUS + AGE + TAX + PTRATIO +
##     cluster_id, data = df)
##
## Residuals:
##     Min      1Q  Median      3Q     Max
## -13.384  -4.245  -1.309   2.503  34.087
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept) 53.894741   3.404083  15.832  < 2e-16 ***
## CRIM        -0.171059   0.046485  -3.680 0.000259 ***
## ZN           0.017395   0.018637   0.933 0.351101
## INDUS       -0.278733   0.080849  -3.448 0.000614 ***
## AGE         -0.020268   0.016434  -1.233 0.218036
## TAX         -0.001481   0.003344  -0.443 0.658032
## PTRATIO     -1.411251   0.177470  -7.952 1.24e-14 ***
## cluster_id   0.163467   0.413505   0.395 0.692776
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 7.254 on 498 degrees of freedom
## Multiple R-squared:  0.3846, Adjusted R-squared:  0.3759
## F-statistic: 44.46 on 7 and 498 DF,  p-value: < 2.2e-16
```

```
df$resid_m1 = residuals(m1)
df_wgs <- st_transform(df, 4326)

pal_resid <- colorNumeric(
  palette = colorRampPalette(c("blue", "white", "red"))(256),
  domain = df_wgs$resid_m1,
  na.color = "#BDBDBD"
)

leaflet(df_wgs) %>%
  addProviderTiles(providers$CartoDB.PositronNoLabels) %>%
  addCircleMarkers(
    radius = 5,
    stroke = FALSE,
    fillOpacity = 0.8,
    color = ~pal_resid(resid_m1),
    popup = ~paste0(
      "<b>CMEDV :</b> ", round(CMEDV, 2), "<br>",
      "<b>Cluster :</b> ", cluster_id, "<br>",
      "<b>Résidu m1 :</b> ", round(resid_m1, 3)
```

```
    )
  ) %>%
  addLegend(
    position = "bottomright",
    pal = pal_resid,
    values = ~resid_m1,
    title = "Résidus m1",
    opacity = 1
  )
```




```
for (k in c(5, 8, 10, 15)) {
  knn <- knearneigh(coords, k = k)
```

```
  nb <- knn2nb(knn)
  lw <- nb2listw(nb, style = "W")
  print(k)
  print(moran.test(df$resid_m1, lw))
}
```

```
## [1] 5
##
##  Moran I test under randomisation
##
## data:  df$resid_m1
## weights: lw
##
## Moran I statistic standard deviate = 18.012, p-value < 2.2e-16
## alternative hypothesis: greater
## sample estimates:
## Moran I statistic       Expectation          Variance
##       0.467819731      -0.001980198       0.000680310
##
## [1] 8
##
##  Moran I test under randomisation
##
## data:  df$resid_m1
## weights: lw
##
## Moran I statistic standard deviate = 19.003, p-value < 2.2e-16
## alternative hypothesis: greater
## sample estimates:
## Moran I statistic       Expectation          Variance
##      0.3901997697     -0.0019801980      0.0004259046
##
## [1] 10
##
##  Moran I test under randomisation
##
## data:  df$resid_m1
## weights: lw
##
## Moran I statistic standard deviate = 19.444, p-value < 2.2e-16
## alternative hypothesis: greater
## sample estimates:
## Moran I statistic       Expectation          Variance
##      0.3560878086     -0.0019801980      0.0003391332
##
## [1] 15
##
##  Moran I test under randomisation
##
## data:  df$resid_m1
## weights: lw
##
## Moran I statistic standard deviate = 19.864, p-value < 2.2e-16
```

```
## alternative hypothesis: greater
## sample estimates:
## Moran I statistic       Expectation          Variance
##      0.2944212463     -0.0019801980      0.0002226553
```

Modèles spatiaux :

SAR :

```
sar_model <- lagsarlm(fml, data = df, listw = listw)
```

```
summary(sar_model)
```

```
##
## Call:lagsarlm(formula = fml, data = df, listw = listw)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -11.1574  -3.1552  -0.7802   1.7447  29.6244
##
## Type: lag
## Coefficients: (asymptotic standard errors)
##                Estimate  Std. Error z value  Pr(>|z|)
## (Intercept) 19.31456304  3.24534405  5.9515 2.657e-09
## CRIM        -0.07921197  0.03703667 -2.1387   0.03246
## ZN           0.01560632  0.01476548  1.0569   0.29054
## INDUS       -0.14457081  0.06436514 -2.2461   0.02470
## AGE         -0.00046401  0.01303144 -0.0356   0.97160
## TAX         -0.00063168  0.00263957 -0.2393   0.81086
## PTRATIO     -0.57481084  0.14484370 -3.9685 7.233e-05
##
## Rho: 0.72553, LR test value: 187.95, p-value: < 2.22e-16
## Asymptotic standard error: 0.036901
##     z-value: 19.662, p-value: < 2.22e-16
## Wald statistic: 386.58, p-value: < 2.22e-16
##
## Log likelihood: -1622.697 for lag model
## ML residual variance (sigma squared): 33.018, (sigma: 5.7462)
## Number of observations: 506
## Number of parameters estimated: 9
## AIC: 3263.4, (AIC for lm: 3449.3)
## LM test for residual autocorrelation
## test value: 39.364, p-value: 3.5175e-10
```

```
df$res_sar <- residuals(sar_model)
moran.test(df$res_sar, listw)
```

```
##
##  Moran I test under randomisation
##
```

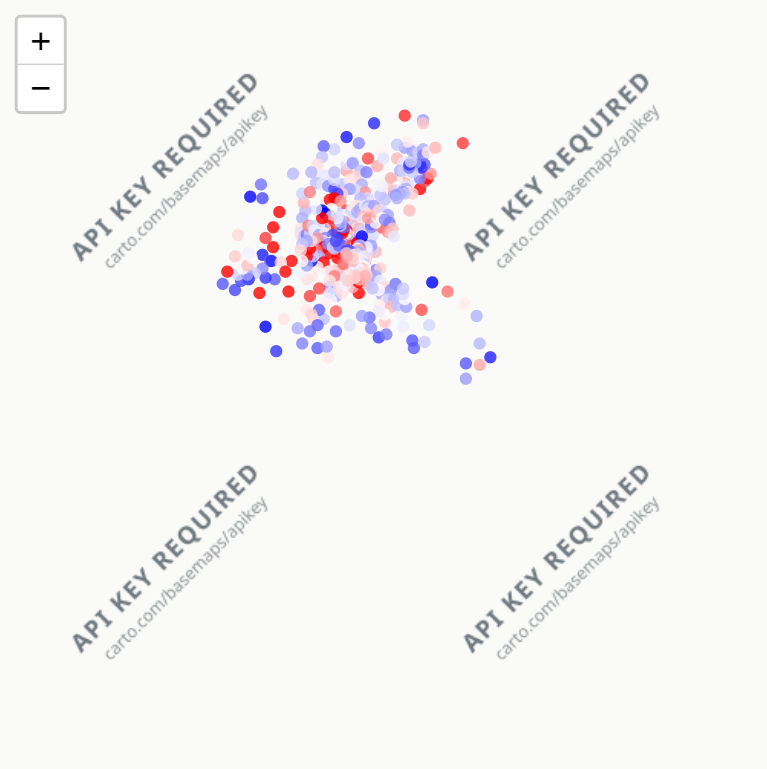
```
## data:  df$res_sar
## weights: listw
##
## Moran I statistic standard deviate = 3.7508, p-value = 8.813e-05
## alternative hypothesis: greater
## sample estimates:
## Moran I statistic       Expectation          Variance
##      0.0754661631     -0.0019801980      0.0004263323
```

```r
m <- st_transform(df, 4326)

lim <- 10
m$resid0_clip <- pmax(pmin(m$res_sar, lim), -lim)

pal <- colorNumeric(
  palette = colorRampPalette(c("blue", "white", "red"))(256),
  domain = c(-lim, lim),
  na.color = "#BDBDBD"
)

leaflet(m) %>%
  addProviderTiles(providers$CartoDB.PositronNoLabels) %>%
  addCircleMarkers(
    radius = 4,
    stroke = FALSE,
    fillOpacity = 0.8,
    color = ~pal(resid0_clip),
    popup = ~paste0(
      "resid0: ", signif(resid0, 4),
      "<br>resid0 (clip): ", signif(resid0_clip, 4)
    )
  ) %>%
  addLegend(
    position = "bottomright",
    pal = pal,
    values = ~resid0_clip,
    title = "Résidus (clip [-4, 4])"
  )
```




```
# 1) IMPORTANT : ellipse => coordonnées cartésiennes (pas lon/lat)
if (isTRUE(sf::st_is_longlat(sf::st_crs(df)))) {
```

```r
  df <- sf::st_transform(df, 3857)
}

# 2) variable à scanner
y <- df$res_sar

# 3) enlever les NA / valeurs non finies
df <- df[is.finite(df$res_sar), ]
y  <- df$res_sar

# 4) IDs directement dans df
df$locid <- seq_len(nrow(df))

# 5) coordonnées
xy <- sf::st_coordinates(df)

td <- file.path(tempdir(), "satscan_sar")
dir.create(td, showWarnings = FALSE, recursive = TRUE)

geo2 <- data.frame(
  locid = df$locid,
  X = xy[, 1],
  Y = xy[, 2]
)

write.table(
  geo2,
  file = file.path(td, "sar.geo"),
  sep = " ",
  row.names = FALSE,
  col.names = FALSE,
  quote = FALSE
)

readLines(file.path(td, "sar.geo"), n = 5)
```

```
## [1] "1 -7898674.46923673 5199254.07468672"
## [2] "2 -7898117.87178276 5204143.3133027"
## [3] "3 -7896559.39891165 5203466.1914073"
## [4] "4 -7895668.84298531 5204970.97242127"
## [5] "5 -7895000.92604055 5205723.45252267"
```

```r
# 6) forcer le séparateur décimal à "."
options(OutDec = ".")

# 7) fichier cas : LocationID, n, y
cas <- data.frame(
  locid = df$locid,
  n = 1,
  y = df$res_sar
)

write.table(
```

```
  cas,
  file = file.path(td, "sar.cas"),
  sep = " ",
  row.names = FALSE,
  col.names = FALSE,
  quote = FALSE
)

readLines(file.path(td, "sar.cas"), n = 5)
```

```
## [1] "1 1 2.63201411361933" "2 1 -1.024717800908"  "3 1 10.0536646880052"
## [4] "4 1 7.03016051859392" "5 1 10.6894660238086"
```

```
invisible(ss.options(reset = TRUE))

ss.options(list(
  CaseFile = "sar.cas",
  CoordinatesFile = "sar.geo",
  CoordinatesType = 0,                 # 0 = cartésien
  PrecisionCaseTimes = 0,              # pas de temps
  AnalysisType = 1,                    # purely spatial
  ModelType = 5,                       # normal model (continuous)
  ScanAreas = 3,                       # high OR low
  MonteCarloReps = 999,                # B = 999
  MaxSpatialSizeInPopulationAtRisk = 50,
  SpatialWindowShapeType = 1,          # elliptic window
  NonCompactnessPenalty = 0,           # pas de pénalité
  ResultsFile = "sar_scan",
  LogRunToHistoryFile = "n"
))

write.ss.prm(td, "sar_scan")           # -> sar_scan.prm
```

```
invisible(ss.options(reset = TRUE))

ss.options(list(
  CaseFile = "sar.cas",
  CoordinatesFile = "sar.geo",
  CoordinatesType = 0,                 # 0 = cartésien
  PrecisionCaseTimes = 0,              # pas de temps
  AnalysisType = 1,                    # purely spatial
  ModelType = 5,                       # normal model (continuous)
  ScanAreas = 3,                       # high OR low
  MonteCarloReps = 999,                # B = 999
  MaxSpatialSizeInPopulationAtRisk = 50,
  SpatialWindowShapeType = 1,          # elliptic window
  NonCompactnessPenalty = 0,           # pas de pénalité
  ResultsFile = "sar_scan",
  LogRunToHistoryFile = "n"
))

write.ss.prm(td, "sar_scan")           # -> sar_scan.prm
```

```r
summary(sar_scan)        # résumé lisible
```

```
## Purely Spatial analysis
## scanning for clusters with high or low values
## using the Normal model.
##
## Study period.....................: 2000/1/1 to 2000/12/31
## Number of locations..............: 506
## Total number of cases............: 506
## Mean.............................: 0.00000000000000028
## Variance.........................: 33.08
## ----------------------------------------------------------------------------------------------
## There were 4 clusters identified.
## There were 2 clusters with p < .05.
```

```r
names(sar_scan)          # voir les tables dispo
```

```
## [1] "main"       "col"        "rr"         "gis"        "llr"
## [6] "sci"        "shapeclust" "prm"
```

```r
sar_scan$col             # table des clusters (souvent la plus utile)
```

```
##   CLUSTER LOC_ID        X       Y   E_MINOR   E_MAJOR E_ANGLE E_SHAPE      SPAN
## 1       1    371 -7908382 5193150  727.2280  1090.842       0     1.5  1480.911
## 2       2    162 -7911866 5195224  506.6989  1013.398      30     2.0  1487.650
## 3       3    226 -7914949 5190610  953.3041  4766.520     -24     5.0  7405.362
## 4       4    196 -7924723 5198878 3177.9311 15889.656     -84     5.0 15878.395
##   START_DATE   END_DATE NUMBER_LOC      LLR TEST_STAT P_VALUE OBSERVED  MEAN_IN
## 1   2000/1/1 2000/12/31          5 51.68785  51.68785   0.001        5 24.72529
## 2   2000/1/1 2000/12/31          5 25.71454  25.71454   0.012        5 17.88129
## 3   2000/1/1 2000/12/31          6 15.84109  15.84109   0.521        6 12.92280
## 4   2000/1/1 2000/12/31          7 11.99356  11.99356   0.944        7 10.43917
##     MEAN_OUT VARIANCE      STD GINI_CLUST
## 1 -0.2467594 26.97045 5.193309      FALSE
## 2 -0.1784560 29.88639 5.466845      FALSE
## 3 -0.1550736 31.07578 5.574566      FALSE
## 4 -0.1464413 31.55198 5.617115      FALSE
```

```r
sar_scan$sci
```

```
##   CLUSTER DATASET CATEGORY OBSERVED  MEAN_IN   MEAN_OUT VARIANCE      STD
## 1       1       1        1        5 24.72529 -0.2467594 26.97045 5.193309
## 2       2       1        1        5 17.88129 -0.1784560 29.88639 5.466845
## 3       3       1        1        6 12.92280 -0.1550736 31.07578 5.574566
## 4       4       1        1        7 10.43917 -0.1464413 31.55198 5.617115
```

```r
gis_sar <- sar_scan$gis

loc_col <- names(gis_sar)[grepl("loc", names(gis_sar), ignore.case = TRUE) &
                          grepl("id", names(gis_sar), ignore.case = TRUE)][1]
```

```r
clu_col <- names(gis_sar)[grepl("cluster", names(gis_sar), ignore.case = TRUE)][1]

if (is.na(loc_col) || is.na(clu_col)) {
  stop("Je ne trouve pas les colonnes locid/cluster dans res$gis. Fais names(res$gis) pour voir les noms
}

df$cluster_sar <- gis_sar[[clu_col]][match(df$locid, gis_sar[[loc_col]])]
df$cluster_sar[is.na(df$cluster_sar)] <- 0L

table(df$cluster_sar)
```

```
##
##   0   1   2   3   4
## 483   5   5   6   7
```

```r
df$cluster_sar <- factor(df$cluster_sar)
levels(df$cluster_sar)
```

```
## [1] "0" "1" "2" "3" "4"
```

```r
sar_1 <- lagsarlm(CMEDV ~ CRIM + ZN + INDUS + AGE + TAX + PTRATIO + cluster_sar,
         data = df, listw = listw)
```

```r
summary(sar_1)
```

```
##
## Call:lagsarlm(formula = CMEDV ~ CRIM + ZN + INDUS + AGE + TAX + PTRATIO +
##     cluster_sar, data = df, listw = listw)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -9.58035 -2.66576 -0.32405  2.06426 22.40754
##
## Type: lag
## Coefficients: (asymptotic standard errors)
##                Estimate Std. Error z value  Pr(>|z|)
## (Intercept)  21.9025073  2.7212659  8.0486 8.882e-16
## CRIM         -0.0706605  0.0282726 -2.4993 0.0124454
## ZN            0.0092378  0.0114607  0.8060 0.4202195
## INDUS        -0.2006423  0.0503042 -3.9886 6.647e-05
## AGE          -0.0197483  0.0100431 -1.9664 0.0492568
## TAX          -0.0032814  0.0020381 -1.6100 0.1073941
## PTRATIO      -0.3956154  0.1141288 -3.4664 0.0005275
## cluster_sar1 28.0302830  2.0750026 13.5086 < 2.2e-16
## cluster_sar2 21.9393813  2.0970986 10.4618 < 2.2e-16
## cluster_sar3 15.3400433  1.8236158  8.4119 < 2.2e-16
## cluster_sar4 12.8283539  1.7554613  7.3077 2.718e-13
##
## Rho: 0.55697, LR test value: 150.55, p-value: < 2.22e-16
## Asymptotic standard error: 0.040217
##     z-value: 13.849, p-value: < 2.22e-16
```

```
## Wald statistic: 191.79, p-value: < 2.22e-16
##
## Log likelihood: -1475.844 for lag model
## ML residual variance (sigma squared): 19.211, (sigma: 4.383)
## Number of observations: 506
## Number of parameters estimated: 13
## AIC: 2977.7, (AIC for lm: 3126.2)
## LM test for residual autocorrelation
## test value: 31.349, p-value: 2.1557e-08
```

```
df$res_sar1 = residuals(sar_1)
```

```
moran.test(df$res_sar1, listw)
```

```
##
##  Moran I test under randomisation
##
## data:  df$res_sar1
## weights: listw
##
## Moran I statistic standard deviate = 4.2931, p-value = 8.812e-06
## alternative hypothesis: greater
## sample estimates:
## Moran I statistic       Expectation          Variance
##      0.0867223802     -0.0019801980      0.0004269141
```

SEM model

```
sem_model <- errorsarlm(
  CMEDV ~ CRIM + ZN + INDUS + AGE + TAX + PTRATIO,
  data = df,
  listw = listw
)

summary(sem_model)
```

```
##
## Call:errorsarlm(formula = CMEDV ~ CRIM + ZN + INDUS + AGE + TAX +
##     PTRATIO, data = df, listw = listw)
##
## Residuals:
##       Min        1Q    Median        3Q       Max
## -13.86085  -2.84376  -0.85998   1.66614  27.30759
##
## Type: error
## Coefficients: (asymptotic standard errors)
##               Estimate Std. Error z value  Pr(>|z|)
## (Intercept) 45.4084238  4.0267914 11.2766 < 2.2e-16
## CRIM        -0.1014299  0.0373976 -2.7122  0.006684
## ZN           0.0399761  0.0178848  2.2352  0.025404
## INDUS       -0.3103601  0.0966210 -3.2121  0.001318
```

```
## AGE         -0.0731702  0.0169789 -4.3095 1.636e-05
## TAX         -0.0126579  0.0038497 -3.2880  0.001009
## PTRATIO     -0.4360102  0.2023902 -2.1543  0.031216
##
## Lambda: 0.85616, LR test value: 245.22, p-value: < 2.22e-16
## Asymptotic standard error: 0.025591
##     z-value: 33.456, p-value: < 2.22e-16
## Wald statistic: 1119.3, p-value: < 2.22e-16
##
## Log likelihood: -1594.062 for error model
## ML residual variance (sigma squared): 28.004, (sigma: 5.2918)
## Number of observations: 506
## Number of parameters estimated: 9
## AIC: 3206.1, (AIC for lm: 3449.3)
```

```
df$res_sem = residuals(sem_model)
```

```
moran.test(df$res_sem, listw)
```

```
##
##  Moran I test under randomisation
##
## data:  df$res_sem
## weights: listw
##
## Moran I statistic standard deviate = 2.0105, p-value = 0.02219
## alternative hypothesis: greater
## sample estimates:
## Moran I statistic       Expectation          Variance
##      0.0395274647     -0.0019801980      0.0004262137
```

```
# 1) IMPORTANT : ellipse => coordonnées cartésiennes (pas lon/lat)
if (isTRUE(sf::st_is_longlat(sf::st_crs(df)))) {
  df <- sf::st_transform(df, 3857)
}

# 2) variable à scanner
y <- df$res_sem

# 3) enlever les NA / valeurs non finies
df <- df[is.finite(df$res_sem), ]
y  <- df$res_sem

# 4) IDs directement dans df
df$locid <- seq_len(nrow(df))

# 5) coordonnées
xy <- sf::st_coordinates(df)

td <- file.path(tempdir(), "satscan_sem")
dir.create(td, showWarnings = FALSE, recursive = TRUE)

geo2 <- data.frame(
```

```
  locid = df$locid,
  X = xy[, 1],
  Y = xy[, 2]
)

write.table(
  geo2,
  file = file.path(td, "sem.geo"),
  sep = " ",
  row.names = FALSE,
  col.names = FALSE,
  quote = FALSE
)

readLines(file.path(td, "sem.geo"), n = 5)
```

```
## [1] "1 -7898674.46923673 5199254.07468672"
## [2] "2 -7898117.87178276 5204143.3133027"
## [3] "3 -7896559.39891165 5203466.1914073"
## [4] "4 -7895668.84298531 5204970.97242127"
## [5] "5 -7895000.92604055 5205723.45252267"
```

```
# 6) forcer le séparateur décimal à "."
options(OutDec = ".")

# 7) fichier cas : LocationID, n, y
cas <- data.frame(
  locid = df$locid,
  n = 1,
  y = df$res_sem
)

write.table(
  cas,
  file = file.path(td, "sem.cas"),
  sep = " ",
  row.names = FALSE,
  col.names = FALSE,
  quote = FALSE
)

readLines(file.path(td, "sem.cas"), n = 5)
```

```
## [1] "1 1 0.283026812558576" "2 1 -1.31239001963811" "3 1 9.19761621926513"
## [4] "4 1 4.93744153625422"  "5 1 9.35997657425975"
```

```
invisible(ss.options(reset = TRUE))

ss.options(list(
  CaseFile = "sem.cas",
  CoordinatesFile = "sem.geo",
  CoordinatesType = 0,                  # 0 = cartésien
```

```
  PrecisionCaseTimes = 0,              # pas de temps
  AnalysisType = 1,                    # purely spatial
  ModelType = 5,                       # normal model (continuous)
  ScanAreas = 3,                       # high OR low
  MonteCarloReps = 999,
  MaxSpatialSizeInPopulationAtRisk = 50,
  SpatialWindowShapeType = 1,          # elliptic window
  NonCompactnessPenalty = 0,
  ResultsFile = "sem_scan",
  LogRunToHistoryFile = "n"
))

write.ss.prm(td, "sem_scan")           # -> sem_scan.prm
```

```
summary(sem_scan)        # résumé lisible
```

```
## Purely Spatial analysis
## scanning for clusters with high or low values
## using the Normal model.
##
## Study period.....................: 2000/1/1 to 2000/12/31
## Number of locations.............: 506
## Total number of cases............: 506
## Mean.............................: -0.00000000000000020
## Variance.........................: 28.06
## ________________________________________________________________________________________
## There were 3 clusters identified.
## There were 2 clusters with p < .05.
```

```
names(sem_scan)          # voir les tables dispo
```

```
## [1] "main"       "col"        "rr"         "gis"        "llr"
## [6] "sci"        "shapeclust" "prm"
```

```
sem_scan$col             # table des clusters (souvent la plus utile)
```

```
##   CLUSTER LOC_ID        X       Y   E_MINOR  E_MAJOR E_ANGLE E_SHAPE      SPAN
## 1       1    371 -7908382 5193150  727.2280 1090.842       0     1.5  1480.911
## 2       2    162 -7911866 5195224  506.6989 1013.398      30     2.0  1487.650
## 3       3    238 -7920282 5191437 1804.2789 9021.395     -12     5.0 14862.746
##   START_DATE   END_DATE NUMBER_LOC      LLR TEST_STAT P_VALUE OBSERVED
## 1   2000/1/1 2000/12/31          5 49.42605  49.42605   0.001        5
## 2   2000/1/1 2000/12/31          5 30.12608  30.12608   0.004        5
## 3   2000/1/1 2000/12/31         13 17.02895  17.02895   0.395       13
##     MEAN_IN   MEAN_OUT VARIANCE      STD GINI_CLUST
## 1 22.314798 -0.2227026 23.07967 4.804131      FALSE
## 2 17.748099 -0.1771267 24.90919 4.990911      FALSE
## 3  8.314299 -0.2192412 26.23263 5.121780      FALSE
```

```
sem_scan$sci
```

```
##   CLUSTER DATASET CATEGORY OBSERVED   MEAN_IN   MEAN_OUT VARIANCE      STD
## 1       1       1        1        5 22.314798 -0.2227026 23.07967 4.804131
## 2       2       1        1        5 17.748099 -0.1771267 24.90919 4.990911
## 3       3       1        1       13  8.314299 -0.2192412 26.23263 5.121780
```

```
gis_sem <- sar_scan$gis

loc_col <- names(gis_sem)[grepl("loc", names(gis_sem), ignore.case = TRUE) &
                          grepl("id", names(gis_sem), ignore.case = TRUE)][1]
clu_col <- names(gis_sem)[grepl("cluster", names(gis_sem), ignore.case = TRUE)][1]

if (is.na(loc_col) || is.na(clu_col)) {
  stop("Je ne trouve pas les colonnes locid/cluster dans res$gis. Fais names(res$gis) pour voir les noms
}

df$cluster_sem <- gis_sar[[clu_col]][match(df$locid, gis_sar[[loc_col]])]
df$cluster_sem[is.na(df$cluster_sem)] <- 0L

table(df$cluster_sem)
```

```
##
##   0   1   2   3   4
## 483   5   5   6   7
```

```
df$cluster_sem <- factor(df$cluster_sem)
levels(df$cluster_sem)
```

```
## [1] "0" "1" "2" "3" "4"
```

```
sem_m2 <- errorsarlm(
  CMEDV ~ CRIM + ZN + INDUS + AGE + TAX + PTRATIO + cluster_sem,
  data = df,
  listw = listw
)
```

```
summary(sem_m2)
```

```
##
## Call:errorsarlm(formula = CMEDV ~ CRIM + ZN + INDUS + AGE + TAX +
##     PTRATIO + cluster_sem, data = df, listw = listw)
##
## Residuals:
##       Min        1Q    Median        3Q       Max
## -12.23746  -2.53176  -0.40029   1.77356  20.32805
##
## Type: error
## Coefficients: (asymptotic standard errors)
##                Estimate Std. Error z value  Pr(>|z|)
```

```
## (Intercept)  40.3033081  2.9088501 13.8554 < 2.2e-16
## CRIM         -0.0609613  0.0283838 -2.1478 0.0317335
## ZN            0.0321037  0.0134425  2.3882 0.0169296
## INDUS        -0.3229700  0.0713396 -4.5272 5.977e-06
## AGE          -0.0713135  0.0126260 -5.6482 1.622e-08
## TAX          -0.0106888  0.0028618 -3.7350 0.0001877
## PTRATIO      -0.3078189  0.1513414 -2.0339 0.0419579
## cluster_sem1 31.1886517  2.3199662 13.4436 < 2.2e-16
## cluster_sem2 24.6143239  2.2763945 10.8129 < 2.2e-16
## cluster_sem3 13.2687060  1.8079979  7.3389 2.154e-13
## cluster_sem4 14.6707202  1.9217414  7.6341 2.265e-14
##
## Lambda: 0.79988, LR test value: 210.5, p-value: < 2.22e-16
## Asymptotic standard error: 0.032692
##     z-value: 24.467, p-value: < 2.22e-16
## Wald statistic: 598.65, p-value: < 2.22e-16
##
## Log likelihood: -1445.869 for error model
## ML residual variance (sigma squared): 15.995, (sigma: 3.9994)
## Number of observations: 506
## Number of parameters estimated: 13
## AIC: 2917.7, (AIC for lm: 3126.2)
```

```
df$res_sem2 = residuals(sem_m2)
moran.test(df$res_sem2, listw)
```

```
##
##  Moran I test under randomisation
##
## data:  df$res_sem2
## weights: listw
##
## Moran I statistic standard deviate = -0.14663, p-value = 0.5583
## alternative hypothesis: greater
## sample estimates:
## Moran I statistic       Expectation          Variance
##     -0.0050093641     -0.0019801980      0.0004267893
```